Information gathering

The next step is to gather as much information as possible about the underlying application, by going through the following steps:

1. *Errors in responses:* For the attacker, the easiest situation would be to have the results of the modified query displayed as part of the Web server's response. If the Web server (Apache, in this case) is configured to display error messages, a lot of information can be extracted through them. For example, a 403 error page on Apache's website shows *apache/2.2.12 (unix) mod_ssl/2.2.12 openSSL/0.9.7d mod_wsgi/3.2 python/2.6.5rc2 server at httpd.apache.org port 80*. The information includes specific software component information and version numbers, including the version number of Apache. This can be used by attackers to run exploits known to work for a particular version. Moreover, database error messages may also leak information about the table or database structure—for example, an error message saying that some columns have not been grouped, when you inject a *HAVING* clause into a *SELECT* statement.
2. *Guessing the database:* Most of the time, error responses also help in guessing the databases in use. For example, if the Web server is Apache, and the website is built using PHP, then chances are the database is MySQL. If the website is in ASP, then it's likely that it uses an MS SQL Server database. However, a more effective way of distinguishing databases is the table provided by the OWASP Web application security project, which is shown in Figure 4. You can relate keywords in error messages from queries that you inject, with those in Figure 4, to guess the database.
3. *Understanding the query:* It is important to know in what kind of query, and in which part of the query, our injection landed. It could be part of a *SELECT, UPDATE, EXEC, INSERT, DELETE* or *CREATE* statement—or could be part of a sub-query too. Start by determining which field is doing what with your input. For example, in the 'Change Your Password' page, the SQL code may be: *SET password = 'new password' WHERE login = user AND password = 'old password'*. Here, if you inject a new password and a comment character in the 'New Password' field, you may end up changing every password in the table to the one you specified. In the same manner, we can guess a *SELECT* query structure, for example, by looking at the output of *' and '1'='1* and *'and '1'='2*. You can also generate specific errors to determine table and column names, like: *' GROUP BY columnnames HAVING 1=1 --*
   You can inject entire queries after a query termination character, as we saw earlier, to help in determining table names and columns. For MySQL, the query is *SHOW columns FROM tablename*, while in Oracle it would be *SELECT * FROM tab_column WHERE table_name='tablename'*. Similarly, DB2, Postgres, etc, have their own syntax.
4. *Time to penetrate:* Once basic information about the database, the query structure and privileges is known, the penetration is started. Extracting data is easy once the database has been enumerated, and the query is understood. For example, to get the password for the login name *admin* we would try the malicious URL: *http://www.example.com/products.asp?id=0;UNION SELECT TOP 1 password FROM account_table WHERE login_name='admin'--* . The same applies to any other specific login name. You can also extract a password from hashes by converting the hashes kept in binary form to a hex format, and that can be displayed as part of an error message.

We have covered a lot about SQL injection attacks so far, but if you want to know more, explore the references at the end of the article. Now, the biggest question left to be answered is: How do we make our present Web applications attack-proof—or at least, as hostile as possible to malicious SQL injectors? The answer is, carefully follow these tips given below:

1. The best way to check whether your website and applications are vulnerable to SQL injection attacks is by using automated and heuristic Web vulnerability scanners (see the 'Web App Security Checking' box). These can also check for cross-site-scripting attacks, and many more attacks that we will be dealing with in later articles.
2. Input validation is the most important part of defending yourself against SQL injection. If the input is supposed to be numeric, use a separate variable in your server-side scripts to store it, and reject bad input, rather than attempting to escape or modify it.
3. Implement filters against keywords like *"select", "insert", "update", "shutdown", "delete", "drop",* and special characters like *"--", "'".* Also, don't do these validations in Javascript/on the client side; rather, do such filtering at the server side itself. Not only will smart attackers figure out how to bypass your Javascript, they will also learn exactly what special filters you have, when they view the Javascript.
4. Try to use encrypted cookies, and not to store values in hidden fields.
5. Make sure you run database services as a low-privilege user account. Remove unused stored procedures and functionality, or restrict access to those, to administrators. Also change permissions and remove *"public"* access to system objects. If the application only requires read access to certain tables, then the account for that application must be limited to read access only. Don't forget to firewall the server so that only trusted clients can connect to it.